

Document Version: 1.0 (a newer version number means an update on the project)
Assignment Date: February 13, 2025, 5:30pm
Due Date: **March 3, 2025, 11:59pm**

Objectives:

- Continue improving your Go skills
- Exercise with distributed client/server communication using socket programming
- Exercise implementing concurrency, shared memory communication, distributed memory communication, and multiplexing via goroutines, TCP sockets, channels, and `select` in Go
- Continue experimenting with AWS

Project Description

Create four `t2.micro` AWS EC2 instances with Ubuntu Server 24.04 LTS and install Go on them. Dedicate one instance as the Server and install the “Chat Server” program (`ch8/chat`) from our reference Go book (Donovan & Kernighan) on it, and the other three instances as Clients and install the “Chat Client” program (`ch8/netcat3`) on them. Source code for these programs is available in our Go reference book’s code repository (<https://github.com/adonovan/gopl.io/>). Finally, authorize inbound traffic for your Server instance and complete the following three parts:

Part 1 (20 pts): Interface and Configuration

Modify the Server code so that it accepts a `PORT` number as a command line argument, and listens on this particular `PORT`. Modify the Client code so that it accepts the following two command line arguments:

- The *first argument* is the user name of the Client (`alice`, `bob`, `eve`, etc.). The Server must use this user name as a prefix to each message from the Client. The Server should not give out Clients’ IP addresses or domain names when broadcasting messages since this is considered privileged information and should be known only to the Server.
- The *second argument* is the IP address and `PORT` number of the Server in the format `IP:PORT`. Note that your AWS instances have a private non-routable IP assigned to the instance’s virtual network adapter, and a public routable IP mapped to your instance by the AWS infrastructure. Use the public IP for your Server as shown in the related lecture video, which is listed on the AWS Console. Choose any port number you wish, but it must be in the range 1024–65535 (inclusive) since ports less than 1024 are privileged and require root permissions.

Run the Server and three Clients in different instances by passing the correct arguments as described above. The chat program should allow the Clients located in different instances to group chat with each other in a distributed setting with the help of a centralized Server located in a different instance from the Clients.

Part 2 (30 pts): Server announcements

Modify the Server so that the `broadcaster` announces the names of current set of clients to each newly arriving Client. This requires that the `clients` set, as well as the `entering` and `leaving` channels record the client name.

Part 3 (50 pts): Private messages

Modify the Server so that it accepts a special message from the Clients indicating a *private* message that should be delivered only to the particular recipient that the sender specifically named in the message (as opposed to being a group message that is broadcast to every Client). Design your own communication protocol facilitating

this private message functionality (define your unique communication rules such as the message format, etc.) and describe your design in your report.

Collaboration and Code Sharing Policy

This course assumes that assignments submitted by students will be generated by the students themselves working individually. This policy indicates the following constitute violations of academic honesty: a student has another person/entity do the work of any portion of a graded assignment for them, which includes purchasing work from a company, hiring a person or company to complete an assignment or exam, **and/or fully/partially copying code from other resources (such as Internet) or generating code using generative AI tools (such as ChatGPT)**. Except parts provided by us or the book, if any, all submitted code must be written by *you* and must not be shared. You are not allowed to go over your classmates' or friends' code, and they are not allowed to see your code. You are only allowed to have high-level verbal discussions about the project with other students. In addition, you are not allowed to publish your code partially (in forums or other sites for asking questions) or fully (in public version control services such as GitHub, etc.). **You cannot post your code on your public GitHub account, unless you make it private!** You are also not allowed to share your code with future students of this course. **You may NOT use Chegg.com, CourseHero.com, MyAssignmentHelp.com, or any other sites to post your assignment/code or receive any help.** Not complying with these collaboration and code sharing rules will put you and those with whom you share your code under the risk of plagiarism for this or the following semesters. Cheating/copying of assignments (including Internet resources) will be reported to the dean's office for plagiarism and a grade of F will be recorded for the course. **Just check with the instructor in advance if you are not sure if a certain practice would put you under the risk of plagiarism. A simple MS Teams chat with the instructor will suffice!**

Submission

Submission is through Blackboard, strictly following the instructions below. Your work will be penalized 5 points out of 100 if the submission instructions are not followed.

What to Submit

1. Go source files used to solve the project.
2. `report.pdf` including the following information (write the question first, then your answer underneath):

Question 1: Your name and ID.

Question 2: Did you fully implement the project as described? If not, what parts are not implemented at all, or what parts are not implemented as specified in the project description?

Question 3: For each part, briefly describe how you accomplished the task, and share any details that is asked from you in the project description, such as *screenshots, plots, etc.*, to prove your work.

Be sure to answer any additional questions in the project description above.

How to Submit

1. Create a directory and name it as the combination of the project number and your UofL student ID number. For example, if a student's ID is 1234567, then the name of the directory will be `ProjectX-1234567` for project X.
2. Put all the files to be submitted (only the ones asked in the *What to Submit* section above) into this directory.

3. Zip the directory. As a result, you will have the file `ProjectX-1234567.zip`.
4. Upload the file `ProjectX-1234567.zip` to Blackboard using the "Attach File" option. You do not need to write anything on the "Submission" and "Comments" sections.
5. You are allowed to make multiple submission attempts but only the last attempt submitted before the deadline will be graded. **NO LATE SUBMISSIONS WILL BE GRADED!**

1 Grading

Grading of your program will be done by an automated process. Your program will also be passed through a copy-checker program which will examine the source codes if they are original or copied; we have a database of similar solutions from previous years as well as from the Internet. We will also examine your source file(s) manually.

Updates

- No updates.